

Agentic SRA Framework — Use Cases

Document: UC-FW-001 · **Version:** 1.0 · **Status:** Approved (2026-09-10) **Audience:** Cybersecurity engineers, regulatory reviewers, capstone advisor

1. Purpose

This document describes who uses the Agentic SRA Framework, what they need it to do, and the end-to-end flows they walk through. It is written for the framework itself, not for the PMx-100 reference device — the PMx-100 corpus is one *input* to the flows described here.

2. Scope

The framework helps a cybersecurity engineer produce an FDA-ready Security Risk Assessment (SRA) for a legacy connected medical device. Input is a GitHub repo containing the device's docs (SAD, SRS, SDD, SDS, threat model, manuals) and firmware source. Output is a per-threat SRA with CVSS scoring, gap analysis against FDA 2023 / IEC 62443 / AAMI TIR57 / NIST 800-193, and concrete proposed changes to documents and code that the engineer reviews and signs off.

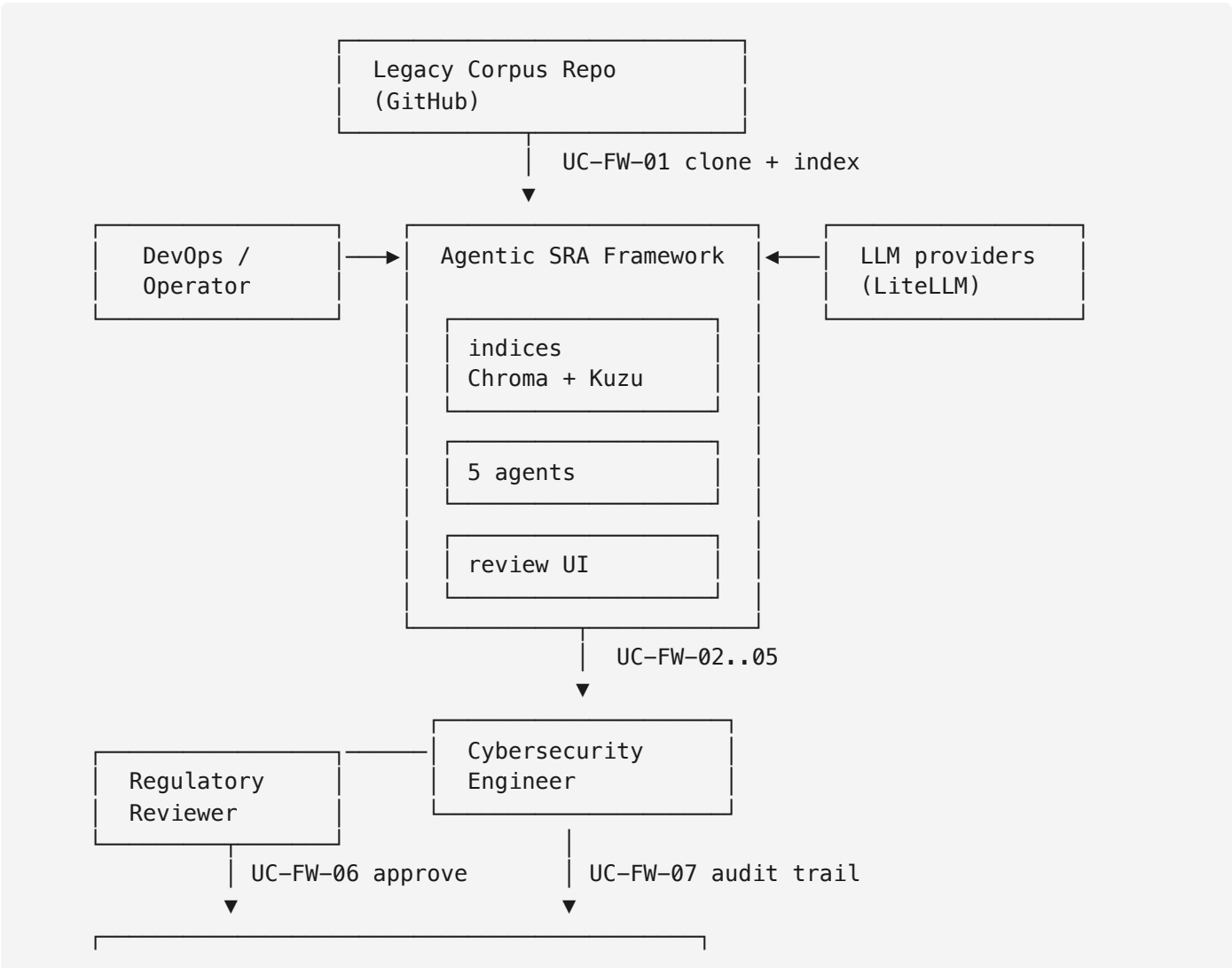
The framework is **not** a decision authority. Every proposal goes through a human review step; the engineer owns the final content.

3. Actors

Actor	Description	Trust level
Cybersecurity Engineer	Primary operator. Runs the framework, reviews per-threat proposals, edits or rejects entries, signs off the draft.	Full
Regulatory / Senior Reviewer	Approves the final SRA before it is attached to an FDA submission. Reads output, may request revisions.	Full
Framework Operator	Installs the framework, sets up API keys, keeps stores refreshed after corpus updates. Often the same person	Full

Actor	Description	Trust level
(DevOps)	as the engineer for smaller teams.	
Auditor	Reads the output and the provenance trail (commit SHA, model versions, tool calls, review decisions) during an FDA inspection or internal audit.	Read-only
Legacy Corpus Repo (GitHub)	External data source cloned as read-only input. Not a person, but the framework treats it as an external system with a well-defined contract (corpus/<device>/*.*md , `firmware/**/*.*c	*.h`).
LLM Providers	OpenAI (embeddings), plus one of Anthropic / Groq / Gemini / Mistral (agent reasoning). External services reached via LiteLLM.	Untrusted (rate-limited, may fail)

4. Use Case Diagram



5. Use Case Narratives

UC-FW-01 · Onboard a new device corpus

Actor: Framework Operator **Trigger:** A cybersecurity uplift effort begins for a device that already has a legacy corpus in a GitHub repository. **Preconditions:**

- Framework installed, `.env` contains at least `OPENAI_API_KEY` and one agent-LLM key.
- The legacy corpus repo follows the expected layout (`corpus/<device>/*.md` , `firmware/**/*.*c|*.h`).
- Network access to GitHub and to the LLM providers.

Main flow:

1. Operator runs `sra init --repo <URL> --device <name>` .
2. Framework git-clones the repo into `input/<device>/` and writes `.manifest.json` capturing commit SHA, clone timestamp, file count, and total size.
3. Operator runs `sra index --device <name>` .
4. Framework builds the vector index (Chroma) by chunking every doc and firmware function, embedding with OpenAI `text-embedding-3-large` , and writing to `stores/chroma/` .
5. Framework produces seven graph-extract JSON files, then loads them into a Kuzu property graph at `stores/kuzu/<device>.kuzu` .
6. Framework prints per-source counts, dangling-edge report, and estimated embedding cost. The corpus is ready.

Success criteria:

- All expected chunks present in Chroma (spot-checked via `sra mcp-test`).
- Kuzu graph responds to a sanity Cypher query for a known threat.
- `sra status --device <name>` shows the manifest and cache summary.

Alternate flows:

- **A1. Repo already cloned.** `sra init` refuses without `--force` ; operator adds `--force` to replace.

- **A2. LLM provider unavailable.** Embedder retries with exponential backoff (up to 5 attempts); on final failure the run aborts, prints which batches succeeded so the operator can resume.
 - **A3. Corpus deviates from expected layout.** Chunker reports the missing files but continues with what it can find; the SRA later flags the gaps.
-

UC-FW-02 · Generate an SRA draft for a device

Actor: Cybersecurity Engineer **Trigger:** Indexed corpus is ready and the engineer wants a first-pass SRA. **Preconditions:**

- UC-FW-01 has completed successfully for the target device.
- Agent-LLM API key is valid and has sufficient quota.

Main flow:

1. Engineer runs `sra draft --device <name>`.
2. Framework spawns the three MCP servers (`corpus-vector` , `corpus-graph` , `corpus-files`) as stdio subprocesses.
3. LangGraph pipeline runs, five agents in sequence per threat:
 - Ingestion agent enumerates threats from the graph.
 - Compliance mapper reads the threat and the relevant clauses, decides per-clause verdict (present / partial / missing).
 - Code analysis agent walks the graph to find implicated code, uses `grep/read` to confirm or refute the control's presence.
 - Threat/Control/Risk agent composes the 12-field SRA entry with CVSS v3.1, proposed control, proposed doc + code changes, priority, and effort estimate.
 - Report generator writes each entry to `output/<device>_sra_draft.md` and `.json` as it finalizes, so a crash mid-run keeps the partial output.
4. Framework prints per-threat progress, per-agent LLM call counts, running token cost, and estimated time-to-completion.
5. On completion, framework prints the path to the draft and the audit log.

Success criteria:

- One SRA entry per threat processed.
- Every entry cites specific corpus items (SRS ids, code file:line, clause ids).
- Total LLM cost within the engineer's declared budget.

Alternate flows:

- **A1. Engineer wants a subset.** `--limit 20` processes the top-20 priority threats; `--category OTA` filters by threat category; `--threat T-OTA-P01-T` runs a single threat.
 - **A2. LLM rate-limited.** Framework backs off and continues; log records the retry.
 - **A3. Agent produces malformed JSON.** Report generator validates against schema; on failure the entry is written with a `parse_error` flag so the engineer can fix it manually rather than losing the LLM's reasoning.
-

UC-FW-03 · Review and refine SRA entries

Actor: Cybersecurity Engineer **Trigger:** A draft file exists at `output/<device>_sra_draft.md`.

Preconditions: UC-FW-02 has produced a draft.

Main flow:

1. Engineer runs `sra review --device <name>`.
2. Framework starts a Streamlit UI on `http://localhost:8501` and opens it in the browser.
3. UI presents each SRA entry as a card with: threat title + STRIDE, CVSS, existing controls, gap analysis, proposed controls, proposed doc changes, proposed code diffs, and buttons **Approve** / **Edit** / **Reject** / **Defer**.
4. For each entry the engineer chooses an action:
 - **Approve** — entry copied verbatim into `output/<device>_sra_final.json`.
 - **Edit** — inline editors let the engineer change any field; saved edits go into the final file with `edited_by: <user>`.
 - **Reject** — entry excluded from the final file, with a required rejection reason recorded for the audit trail.
 - **Defer** — kept in a "needs more info" bucket; the engineer can come back later.
5. UI shows running progress (X of 225 threats reviewed).
6. Engineer may pause and resume; state is persisted in `output/<device>_review_state.json`.

Success criteria:

- Every threat has an explicit disposition (approved / edited / rejected / deferred).
- Every decision has a timestamp and reviewer identity in the audit log.

Alternate flows:

- **A1. Engineer wants a colleague to review a subset.** UI supports assigning entries to other reviewers via a comment field; the state file records who reviewed what.

- **A2. Engineer needs deeper evidence.** UI has an "Ask the graph" button that runs a live Cypher query (via `corpus-graph` MCP) to show related items without leaving the review flow.
-

UC-FW-04 · Export the final SRA for submission

Actor: Cybersecurity Engineer or Regulatory Reviewer **Trigger:** Review is complete for every threat. **Preconditions:** UC-FW-03 has reached 100 % dispositioned.

Main flow:

1. Actor runs `sra export --device <name>`.
2. Framework produces:
 - `output/<device>_sra_final.md` — human-readable final SRA
 - `output/<device>_sra_final.pdf` — via pandoc + Chrome headless
 - `output/<device>_sra_final.docx` — Word format for reviewer markup (change-tracking, comments)
 - `output/<device>_sra_final.json` — machine-readable form for downstream tooling
 - `output/<device>_provenance.json` — commit SHA, model ids used, per-agent token counts, review decisions, timestamps
3. Framework prints file paths and a one-page summary (threat counts by disposition, by priority, by residual risk).

Success criteria: PDF renders without error, JSON validates against the SRA schema, provenance file is complete.

UC-FW-05 · Refresh the SRA after corpus updates

Actor: Cybersecurity Engineer **Trigger:** Firmware, docs, or threat model in the legacy corpus have been updated and pushed to GitHub. **Preconditions:** A prior SRA (draft or final) exists.

Main flow:

1. Engineer runs `sra refresh --device <name>`.
2. Framework `git pull`s the corpus, re-runs indexing (Chroma incremental update, Kuzu rebuild), and compares the new graph against the old.

3. Framework identifies affected threats (any threat whose linked SRS / SDS / code node changed).
4. Framework runs the agents *only* for affected threats and produces `output/<device>_sra_delta.md` — a diff view showing the changed entries against the previous final SRA.
5. Engineer reviews the delta in UC-FW-03 (delta-only mode), approves changes, exports (UC-FW-04).

Success criteria: Unaffected entries carry over unchanged; affected entries clearly marked; provenance updated with new commit SHA.

UC-FW-06 • Interactive investigation of a specific threat or vulnerability

Actor: Cybersecurity Engineer (deep-dive mode) **Trigger:** During review the engineer wants to understand a threat before deciding. **Preconditions:** Indices exist for the target device.

Main flow:

1. Engineer either uses the review-UI "Ask the graph" button or launches a standalone MCP session: `sra mcp graph --device <name>` (or `vector`, or `files`).
2. Engineer asks natural-language questions or Cypher queries — e.g. "which controls satisfy FDA-2023-§V.B.3 and are implemented by any SRS item that traces to T-OTA-P01-T?"
3. Framework returns the graph rows or vector hits; the engineer decides how to act.
4. Any resulting edits happen in UC-FW-03.

Success criteria: Every query answered from the local indices (no external calls); results are traceable back to source files.

UC-FW-07 • FDA audit — reconstruct provenance

Actor: Auditor (FDA reviewer or internal quality) **Trigger:** FDA requests evidence of how a claim in the SRA was reached. **Preconditions:** Signed-off SRA exists with its provenance file.

Main flow:

1. Auditor opens `output/<device>_provenance.json` and the sibling `output/<device>_prompts.jsonl` file (line-delimited log of every LLM interaction).

2. Provenance file shows for every entry: which threat model version (commit SHA), which SRS/SDS/code items were retrieved, which regulatory clauses were checked, which LLM produced the reasoning (model id + version), which reviewer approved it, when.
3. Prompts log records the **exact prompt string, tool-call arguments, and response** for every LLM turn during drafting — one JSON object per line, keyed by entry id and agent name. Auditor can replay the full reasoning chain.
4. Auditor can re-run the framework at that exact commit (`sra init --repo <URL> --commit <SHA>`) to reproduce the draft; deterministic sampling (`temperature=0`) makes outputs closely reproducible modulo provider drift, which is itself recorded in `provenance.model_version`.

Success criteria: Every SRA claim is traceable to (a) a corpus item at a specific commit, (b) the exact LLM prompt that produced the claim, and (c) a reviewer decision by a named person at a specific timestamp.

6. Traceability

Use case	Touches
UC-FW-01	<code>sra init</code> , <code>sra index</code> , Chroma, Kuzu, manifest
UC-FW-02	<code>sra draft</code> , LangGraph, 5 agents, all 3 MCP servers, LLM providers
UC-FW-03	<code>sra review</code> , Streamlit UI, review state, Cypher passthrough
UC-FW-04	<code>sra export</code> , pandoc, Chrome headless, provenance writer
UC-FW-05	<code>sra refresh</code> , delta detection, incremental agents
UC-FW-06	<code>sra mcp <server></code> , standalone MCP session
UC-FW-07	provenance file, reproducibility from commit SHA

7. Out of scope for v1

- Multi-device batch mode. v1 runs one device at a time.
- Web-hosted deployment. v1 is a local CLI + local Streamlit.
- Multi-user concurrent review. v1 assumes one reviewer at a time per device.
- Direct FDA submission portal integration. v1 produces the file; the engineer uploads it manually.
- Continuous monitoring (auto-refresh on GitHub push). v1 is engineer-initiated only.

8. Reviewer decisions (locked v1.0)

Question	Decision
UC-FW-05 (refresh after corpus updates) in scope for capstone?	Yes — in scope for v1
UC-FW-04 export to Word <code>.docx</code> ?	Yes — alongside <code>.md</code> , <code>.pdf</code> , <code>.json</code>
Audit trail (UC-FW-07) — log full LLM prompts?	Yes — full prompts + responses per turn in <code><device>_prompts.jsonl</code>

9. Change log

- **v1.0 (2026-09-10)** — Approved by Cybersecurity Engineer (reviewer). Locked three open questions; expanded UC-FW-04 export list to include `.docx` ; expanded UC-FW-07 to describe the `prompts.jsonl` full-turn log.
- **v0.1 (2026-09-10)** — Initial draft.